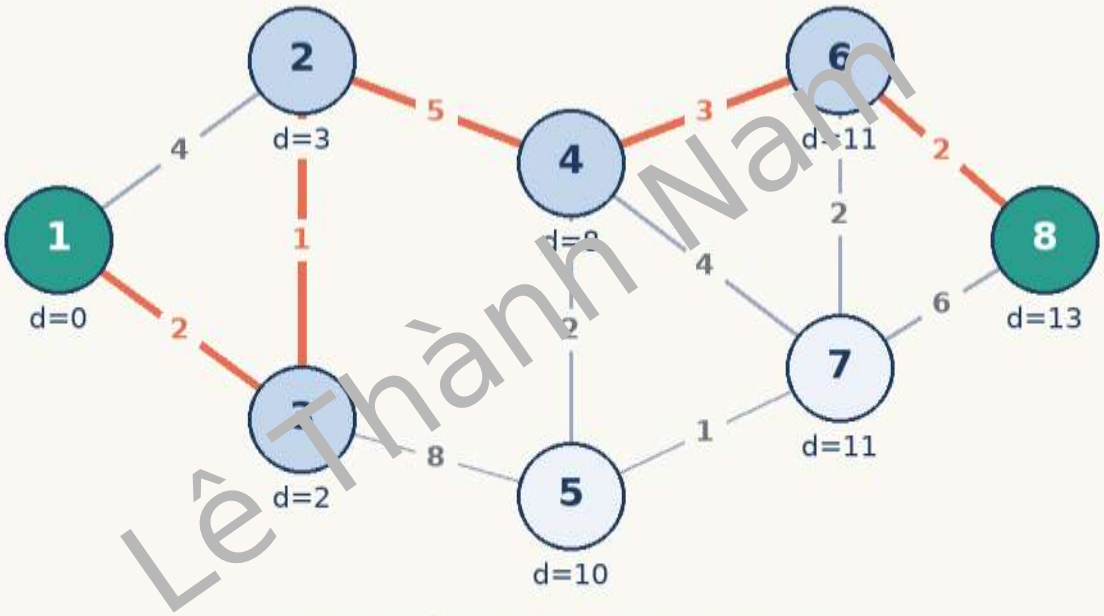
TUYỂN TẬP BÀI TẬP LUYỆN THI TIN HỌC

22 bài CSES Problem Set, dịch tiếng Việt

# THUẬT TOÁN ĐỒ THỊ




*đường ngắn nhất từ 1 tới 8 dài 13*

BFS · DFS | Thành phần liên thông | Đường đi ngắn nhất | DAG

*Ngây thơ → Bế tắc → Phá bế tắc → Tối ưu*


**LÊ THÀNH NAM**

*Tổng hợp và biên soạn*


Lê Thành Nam

# MỤC LỤC

*Ngây thơ → Bế tắc → Phá bế tắc → Tối ưu*

**Giới thiệu** ..... 2

**Chương 1. Duyệt đồ thị: BFS và DFS** ..... 3

**Bài 1.** Đếm số phòng *(CSES 1192)* ..... 4
**Bài 2.** Mê cung *(CSES 1193)* ..... 9
**Bài 3.** Đường truyền tin *(CSES 1667)* ..... 13
**Bài 4.** Chia đội *(CSES 1668)* ..... 17
**Bài 5.** Chuyến đi vòng *(CSES 1669)* ..... 21
**Bài 6.** Quái vật *(CSES 1194)* ..... 25

**Chương 2. Thành phần liên thông** ..... 29

**Bài 7.** Xây đường *(CSES 1666)* ..... 30
**Bài 8.** Xây đường từng ngày *(CSES 1676)* ..... 34
**Bài 9.** Kiểm tra đường bay *(CSES 1682)* ..... 38
**Bài 10.** Các vương quốc *(CSES 1683)* ..... 42

**Chương 3. Đường đi ngắn nhất** ..... 47

**Bài 11.** Đường đi ngắn nhất I *(CSES 1671)* ..... 48
**Bài 12.** Đường đi ngắn nhất II *(CSES 1672)* ..... 52
**Bài 13.** Phiếu giảm giá *(CSES 1195)* ..... 56
**Bài 14.** Điểm cao nhất *(CSES 1673)* ..... 60
**Bài 15.** Tìm chu trình âm *(CSES 1197)* ..... 63
**Bài 16.** Các đường bay rẻ nhất *(CSES 1196)* ..... 67
**Bài 17.** Điều tra đường bay *(CSES 1202)* ..... 71

**Chương 4. Đồ thị có hướng không chu trình (DAG)** ..... 75

**Bài 18.** Lịch học *(CSES 1679)* ..... 76
**Bài 19.** Chuyến đi vòng II *(CSES 1678)* ..... 80
**Bài 20.** Đường bay dài nhất *(CSES 1680)* ..... 84
**Bài 21.** Đếm đường chơi *(CSES 1681)* ..... 88
**Bài 22.** Nhặt xu *(CSES 1686)* ..... 92

Lê Thành Nam

# GIỚI THIỆU

## Tài liệu này dành cho ai

Tài liệu gồm 22 bài của mục Graph Algorithms trong CSES Problem Set (cses.fi/problemset), được sắp lại thành 4 chương theo kĩ thuật: *Duyệt đồ thị BFS và DFS, thành phần liên thông, đường đi ngắn nhất (Dijkstra, Bellman-Ford, Floyd–Warshall), và đồ thị có hướng không chu trình (DAG)*. Tài liệu phù hợp với học sinh đã biết lập trình cơ bản, đã làm quen với sắp xếp và quy hoạch động, đang luyện thi học sinh giỏi cấp tỉnh và các kì thi tương đương.

Mỗi bài gồm hai phần. Đề bài được viết lại bằng tiếng Việt, giữ đúng yêu cầu, ràng buộc, định dạng vào/ra và ví dụ của đề gốc, kèm bảng subtask do tài liệu bổ sung. Hướng dẫn giải đi qua 4 bước tư duy; mỗi bước có code chạy được, có hình minh họa, bài nào cũng kết thúc bằng bảng so sánh các tầng lời giải đo thật trên máy.

## 4 bước tư duy giải một bài

| Bước | Nội dung |
|---|---|
| **Ngây thơ** | Ý tưởng tự nhiên nhất, làm đúng như đề mô tả; có code và độ phức tạp. |
| **Bế tắc** | Biểu đồ thời gian đo thật, chỉ ra chỗ chậm hoặc chỗ sai, kết thúc bằng một câu hỏi then chốt. |
| **Phá bế tắc** | Nhận xét hoặc tính chất giúp vượt qua chỗ tắc, có ví dụ và hình minh họa. |
| **Tối ưu** | Code cuối cùng, độ phức tạp và các lưu ý: kiểu dữ liệu, tràn số, đọc ghi nhanh. |

## Subtask và bảng so sánh các tầng

Đề CSES không chia subtask. Tài liệu bổ sung cho mỗi bài ba mức: Subtask 1 (30%) là ràng buộc nhỏ để cách ngây thơ lấy điểm, Subtask 2 (30%) và Subtask 3 (40%) là ràng buộc gốc của CSES hoặc ràng buộc mở rộng (thường $n \le 10^6$, $m \le 2 \cdot 10^6$). Ở nhiều bài, Subtask 3 dùng đồ thị rất "sâu" (một chuỗi hàng triệu đỉnh) để phân biệt cách viết đệ quy với cách viết không đệ quy.

Cuối mỗi bài là bảng so sánh: mỗi tầng code chạy trên test lớn nhất của từng subtask, ghi thời gian thật và số điểm. Một tầng chỉ được điểm của subtask nếu cho kết quả đúng trong không quá 1 giây, dùng không quá 512 MB bộ nhớ và 8 MB ngăn xếp (ô "lỗi" là chương trình bị dừng vì tràn bộ nhớ hoặc tràn ngăn xếp). Thời gian đo bằng g++ 13 (tùy chọn -O2) trên máy chủ Linux, tính cả đọc và ghi dữ liệu; trên máy chấm khác các con số có thể chênh lệch đôi chút.

## Sử dụng tài liệu

Với mỗi bài, hãy tự suy nghĩ ít nhất 20–30 phút trước khi đọc hướng dẫn. Khi đọc, dừng lại ở câu hỏi then chốt cuối phần **Bế tắc** và tự trả lời trước khi xem phần **Phá bế tắc**.

Mọi code trong tài liệu đã được biên dịch, chạy đúng ví dụ của đề, và đối chiếu với lời giải tham chiếu (hoặc bộ kiểm tra đáp án) trên 200–300 test ngẫu nhiên.

Lê Thành Nam

# Chương 1. Duyệt đồ thị: BFS và DFS

**Trọng tâm chương**

Đồ thị gồm các đỉnh và các cạnh nối chúng; lưới ô vuông cũng là một đồ thị (mỗi ô là một đỉnh). Hai cách duyệt cơ bản là BFS (theo chiều rộng, dùng hàng đợi) và DFS (theo chiều sâu, dùng ngăn xếp hoặc đệ quy). BFS cho đường đi ít cạnh nhất; DFS giúp phát hiện chu trình. Chương này cũng nhấn mạnh một lỗi rất hay gặp: đệ quy quá sâu làm tràn ngăn xếp.

## Kiến thức cần nhớ

- Biểu diễn đồ thị. Danh sách kề: adj[u] chứa các đỉnh kề u; với hàng triệu cạnh nên dùng dạng mảng phẳng (CSR).
- BFS. Hàng đợi; đánh dấu đã thăm ngay khi đưa vào hàng đợi; khoảng cách tới đỉnh mới = khoảng cách đỉnh cũ + 1.
- Truy vết đường đi. Lưu par[v] (hoặc hướng đi vào v), đi ngược từ đích về nguồn rồi đảo lại.
- DFS và chu trình. Ở đồ thị vô hướng, gặp một đỉnh đã thăm khác cha là có chu trình.
- Ngăn xếp 8 MB. Đệ quy sâu khoảng $10^5$–$10^6$ tầng có thể bị lỗi; khi đó viết DFS bằng ngăn xếp tự quản lý hoặc dùng BFS.

## Các bài trong chương

| Bài | Tên bài | Mã CSES | Kĩ thuật chính |
|---|---|---|---|
| 1 | Đếm số phòng | 1192 | BFS trên lưới |
| 2 | Mê cung | 1193 | BFS, truy vết |
| 3 | Đường truyền tin | 1667 | BFS, truy vết |
| 4 | Chia đội | 1668 | Tô hai màu |
| 5 | Chuyến đi vòng | 1669 | DFS, cạnh ngược |
| 6 | Quái vật | 1194 | BFS nhiều nguồn |

Lê Thành Nam

# Bài 1. Đếm số phòng

**CSES 1192 – Counting Rooms** Thời gian: 1 giây · Bộ nhớ: 512 MB
Đề gốc: https://cses.fi/problemset/task/1192

## Đề bài

Cho bản đồ một tòa nhà dạng lưới $n \times m$; mỗi ô là sàn (“.”) hoặc tường (“#”). Từ một ô sàn có thể đi sang ô sàn kề cạnh (trên, dưới, trái, phải).

Một phòng là một vùng các ô sàn đi lại được với nhau. Hãy đếm số phòng.

### Dữ liệu vào

Dòng đầu chứa hai số nguyên $n$ và $m$.

$n$ dòng tiếp theo, mỗi dòng gồm $m$ kí tự “.” hoặc “#”.

### Kết quả

Một số nguyên là số phòng.

### Ví dụ

| Dữ liệu vào | Kết quả |
|---|---|
| 5 8<br>########<br>#..#...#<br>####.#.#<br>#..#...#<br>######## | 3 |

### Subtask

| Subtask | Ràng buộc | Điểm |
|---|---|---|
| Subtask 1 | n, m ≤ 50 | 30 |
| Subtask 2 | n, m ≤ 1 000 (ràng buộc gốc) | 30 |
| Subtask 3 | n, m ≤ 2 000 | 40 |

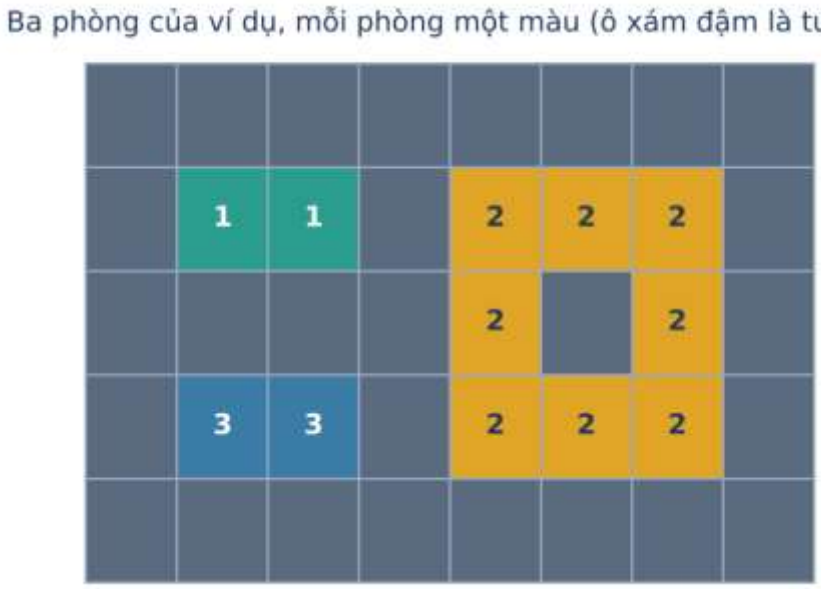
Lê Thành Nam

## Hướng dẫn giải

### NGÂY THƠ Cho các ô “truyền nhãn” cho nhau

Coi mỗi ô sàn là một đỉnh của đồ thị, hai ô sàn kề cạnh được nối bằng một cạnh. Một phòng chính là một thành phần liên thông của đồ thị này. Hình 1.1 tô ba phòng của ví dụ bằng ba màu.

Cách nghĩ đơn giản nhất: Ban đầu mỗi ô sàn mang một nhãn riêng (số thứ tự của ô). Lặp lại nhiều lượt: Mỗi ô nhìn bốn ô kề, nếu thấy nhãn nhỏ hơn thì lấy nhãn đó. Khi không còn ô nào đổi nhãn, các ô cùng phòng mang chung nhãn nhỏ nhất của phòng, và số phòng là số ô giữ đúng nhãn của chính mình.




*Hình 1.1. Ba phòng của ví dụ.*

**Code.** Tầng 1. Lan nhãn tới khi ổn định

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n, m;
    cin >> n >> m;
    vector<string> g(n);
    for (auto &s : g) cin >> s;
    // Mỗi ô sàn mang một nhãn riêng; lặp lại: mỗi ô lấy nhãn nhỏ nhất của các ô sàn
kề nó
    vector<vector<int>> lab(n, vector<int>(m, -1));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (g[i][j] == '.') lab[i][j] = i * m + j;
    int dr[4] = {1, -1, 0, 0}, dc[4] = {0, 0, 1, -1};
    bool changed = true;
    while (changed) {                          // dừng khi không nhãn nào đổi nữa
        changed = false;
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) {
                if (lab[i][j] < 0) continue;
                for (int d = 0; d < 4; d++) {
                    int r = i + dr[d], c = j + dc[d];
                    if (r < 0 || r >= n || c < 0 || c >= m || lab[r][c] < 0) continue;
                    if (lab[r][c] < lab[i][j]) { lab[i][j] = lab[r][c]; changed =
true; }
                }
            }
    }
    int rooms = 0;                             // mỗi phòng có đúng một ô giữ nhãn của
chính nó
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (lab[i][j] == i * m + j) rooms++;
    cout << rooms << '\n';
}
```

Lê Thành Nam

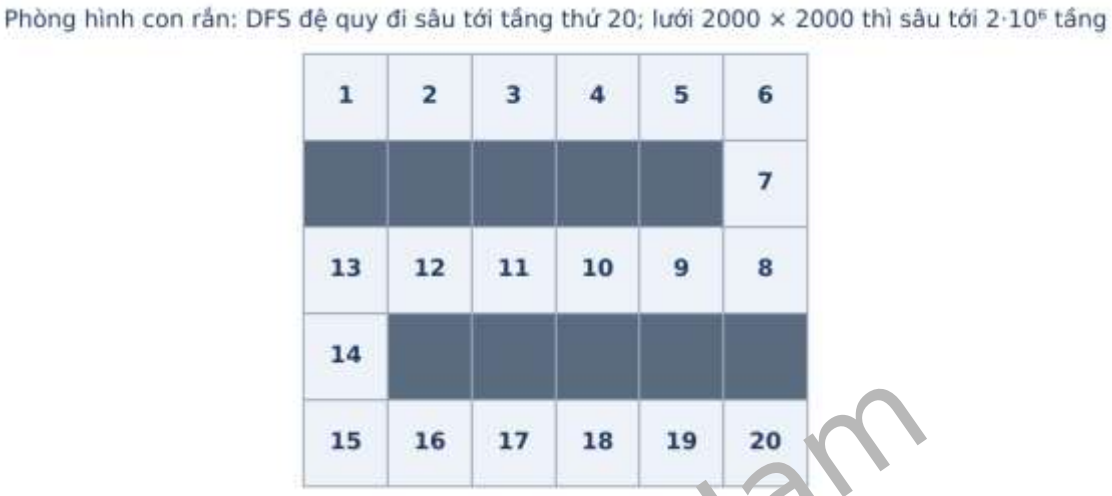
Độ phức tạp: Mỗi lượt quét cả lưới $O(n \cdot m)$, nhưng nhãn nhỏ có thể phải đi qua cả phòng mới tới ô xa nhất, nên số lượt có thể tới $O(n \cdot m)$. Tổng cộng tới $O((n \cdot m)^2)$.

**BẺ TẮC** **Nhãn lan đi quá chậm, còn đệ quy thì quá sâu**

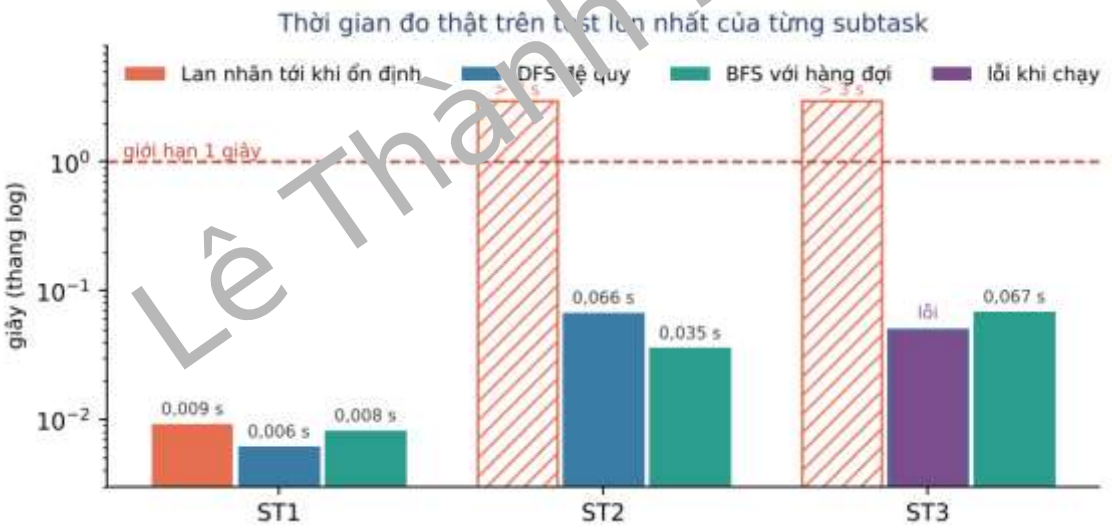
Với lưới 1 000 × 1 000, cách truyền nhãn chạy quá 3 giây (Hình 1.3): Mỗi lượt nhãn chỉ lan được một vài ô, trong khi lượt nào cũng quét lại toàn bộ $10^6$ ô.

Cách tự nhiên hơn là đi khắp phòng bằng DFS (tìm kiếm theo chiều sâu): Gặp một ô sàn chưa thăm thì tăng số phòng, rồi gọi đệ quy sang bốn ô kề để đánh dấu cả phòng. Mỗi ô chỉ được thăm một lần, tổng O(n · m). Với lưới ngẫu nhiên 1 000 × 1 000 chương trình chạy 0,07 giây.

Nhưng với phòng hình con rắn ở Hình 1.2, DFS đi sâu theo một hành lang duy nhất: Lưới 2 000 × 2 000 cần khoảng $2 \cdot 10^6$ lời gọi hàm chồng lên nhau, làm tràn vùng ngăn xếp 8 MB và chương trình bị lỗi. Câu hỏi then chốt: Làm sao đi khắp phòng mà không cần lời gọi đệ quy?




***Hình 1.2.*** *Thứ tự DFS thăm các ô của một phòng hình con rắn.*




***Hình 1.3.*** *Thời gian đo thật của ba tầng lời giải.*

**Code.** Tầng 2. DFS đệ quy

```
#include <bits/stdc++.h>
using namespace std;
int n, m;
vector<string> g;
void dfs(int i, int j) {                    // tô cả phòng chứa ô (i, j)
    if (i < 0 || i >= n || j < 0 || j >= m || g[i][j] != '.') return;
    g[i][j] = '#';                          // đánh dấu đã thăm bằng cách biến thành
tường
    dfs(i + 1, j); dfs(i - 1, j); dfs(i, j + 1); dfs(i, j - 1);
}
int main() {
    cin >> n >> m;
    g.resize(n);
    for (auto &s : g) cin >> s;
    int rooms = 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
```

Lê Thành Nam

```
            if (g[i][j] == '.') { rooms++; dfs(i, j); }   // gặp ô sàn chưa thăm:
phòng mới
    cout << rooms << '\n';
}
```

## PHÁ BẾ TẮC Phá bế tắc: Tự quản lý danh sách ô cần thăm

Đệ quy thực chất là để máy tự nhớ "những ô còn phải xét tiếp". Ta có thể tự nhớ danh sách đó trong một mảng: BFS (tìm kiếm theo chiều rộng) dùng một hàng đợi. Lấy một ô ra khỏi đầu hàng đợi, đưa các ô sàn kề chưa thăm vào cuối hàng đợi, lặp lại cho tới khi hàng đợi rỗng.

Hàng đợi nằm trong một mảng có $n \cdot m$ phần tử, cấp phát trên vùng nhớ thường (rất rộng), nên không còn bị giới hạn 8 MB của ngăn xếp.

## TỐI ƯU BFS với hàng đợi tự viết

Ta đánh dấu một ô là đã thăm ngay khi đưa nó vào hàng đợi (biến nó thành "#"), để một ô không bị đưa vào hai lần. Hàng đợi là một mảng q với hai chỉ số head và tail, mỗi ô được mã hóa thành một số $i \cdot m + j$.

**Code.** Tầng 3. BFS với hàng đợi

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<string> g(n);
    for (auto &s : g) cin >> s;
    vector<int> q(n * m);                  // hàng đợi tự viết, lưu ô dưới dạng i * m
+ j
    int dr[4] = {1, -1, 0, 0}, dc[4] = {0, 0, 1, -1};
    int rooms = 0;
    for (int si = 0; si < n; si++)
        for (int sj = 0; sj < m; sj++) {
            if (g[si][sj] != '.') continue;
            rooms++;                       // phòng mới: BFS tô hết phòng này
            int head = 0, tail = 0;
            q[tail++] = si * m + sj; g[si][sj] = '#';
            while (head < tail) {
                int i = q[head] / m, j = q[head] % m; head++;
                for (int d = 0; d < 4; d++) {
                    int r = i + dr[d], c = j + dc[d];
                    if (r < 0 || r >= n || c < 0 || c >= m || g[r][c] != '.')
continue;
                    g[r][c] = '#'; q[tail++] = r * m + c;
                }
            }
        }
    cout << rooms << '\n';
}
```

Độ phức tạp: $O(n \cdot m)$ thời gian và bộ nhớ.

Bài học: Đếm thành phần liên thông là chạy BFS hoặc DFS từ mỗi đỉnh chưa thăm. Khi đồ thị có thể rất "sâu" (hàng triệu đỉnh trên một đường), hãy dùng BFS hoặc DFS không đệ quy.

Lê Thành Nam